

数据智能 Agent 系统核心改进报告

本报告结合了 `configs/docker.yaml` 中的配置项与系统源码实现，对当前系统的 8 项核心技术改进进行了简要介绍和梳理。由于当前 `docker.yaml` 中禁用了歧义分析节点（`agent.enable_ambiguity_analysis: false`），本报告已剔除所有与歧义分析节点相关的说明，完全契合当前的生产配置。

1. Process Validator（过程验证器）

- 相关配置：`agent.enable_process_validator: true`
- 触发机制：
 - 触发机制一（周期性探查）：结合配置 `process_validator.checkpoint_model_interval: 10`。在 `tool_step` 节点后，若当前已执行的步数与上一次进行过程验证的步数之差达到 10 步，系统将强制路由到验证节点。
 - 触发机制二（终结性验证）：当 Agent 提交最终答案时，系统同样会先路由到 `validate_process` 节点对整条证据链进行最后的严格核验。
- 逻辑控制与重试：
 - 系统调用 `invoke_process_validator` 对比模型的问题、近期执行的步骤日志、语义账本 `semantic_ledger` 及最终答案。
 - 若未通过，且重试次数在配置 `process_validator.retry_limit: 1` 限制内，系统会向 Agent 的 `messages` 历史中注入一个包含具体高置信度风险项（`issues`）和建议执行的下一步探针动作（`required_next_actions`）的 `HumanMessage` 反馈信息，并清空当前答案（`answer = None`），引导主 Agent 重新进行数据探查以校正假设，并重新计算；如果超出重试上限（1次），则予以放行，防止流程卡死。

2. Answer Validator（答案验证器）

- 相关配置：`agent.enable_answer_validator: true`
- 工作原理与触发机制：
 - 在 `validate_answer_step` 节点实现，主要用于在 Agent 完成计算并提交答案（`answer`）时，对最终的数据结构进行语义与格式一致性校验。
- 校验与重试纠偏：
 - 提取答案特征（`answer_fingerprint`），为了加速校验并节省 API 资源，会与历史校验记录（`answer_validation_history`）做缓存对比。若未命中缓存，则通过 `invoke_answer_validator` 对结果数据结构进行严格校验。
 - 纠偏提示：如果校验出数据字段有瑕疵（如列名与题目要求不符、行数超出限制、日期未做 ISO 8601 规范的零填充零对齐（如 `2024-03-01` 误写为 `2024-3-1`）或 DateTime 没有转换到 UTC 时区并以 `Z` 结尾），验证器会拦截该答案，并返回详细的中文反馈消息（`feedback_message`），清空 `answer` 并把控制权交还给主 Agent 提示其修正。
 - 兜底保护：同样受重试限制保护，在多次重试（依据代码中配置的 `validation_retry_limit`）仍未通过时，会强行采纳并接受答案，确保任务最终能正常完结。

3. Knowledge 和 Catalog 注入

- 相关配置: `agent.enable_data_inspector: true`
- 注入机制与流程:
 - **Catalog (数据字典) 注入**: 系统初始化后进入 `global_data_exploration` 节点, 递归分析所有结构化数据源, 并构建包含表结构、字段类型、空值率、基数以及根据配置 `data_inspector.sample_budget.catalog_top_distinct_values: 5` 提取的前 5 个最频繁特征值样例的 `semantic_catalog`。
 - **Knowledge (业务常识) 注入**: knowledge 文档全文随着 catalog 一同注入模型上下文。

4. Agent 工具

在 Starter Kit 中, 为了提高对文本、非结构化常识文档以及结构化数据库表的读取和检索效率, 系统提供并精心优化了如下核心工具链:

- `search_doc` (文档文本检索):
 - **工作原理**: 对应 `search_doc_text` 核心实现。用于在指定的文本文件 (后缀 `.md`、`.txt`、`.rst`) 或全局所有文本中进行正则/关键词匹配。
 - **高级特性**: 支持通过 `context_lines` 获取匹配行前后各若干行的上下文; 并且为了避免文本超大导致 Token 爆炸, 提供了**检索结果分页机制** (可通过 `page` 和 `page_size` 参数进行跳转), 让 Agent 能够优雅且有选择性地搜索背景文档, 替代了模型此前自己手写复杂的 Python 脚本去解析文件的方式。
- `read_doc` (精确章节级文档阅读):
 - **工作原理**: 对应 `read_doc_preview` 实现。支持直接读取文档全文, 更推荐的方案是传入特定的章节标题 (`heading`) 参数只读取该章节。
 - **健壮性优化**: 通过归一化函数 `_normalize_heading_for_match`, 即使 Agent 传入的标题漏掉了前面的序号级标题编号 (如 "2.4.1. 过滤约定" 缩写为 "过滤约定"), 它依然能够智能通过归一化和大小写忽略进行模糊匹配, 提取出当前章节到下一个同级/更高级标题之间的内容, 从而实现了极其精确且省 Token 的文档段落阅读。
- `lookup_doc_outline` (文档目录大纲探索):
 - **工作原理**: 在 Agent 使用 `read_doc` 读取前, 强制或指引它必须先调用此工具来获取文档的所有 Markdown 标题层级 (例如层级 `1` 对应 `#`, 层级 `2` 对应 `##`)。
 - **设计目的**: 在 Agent 拿到整个大纲框架后, 方便其进行有针对性的章节级 `read_doc` 读取, 严厉杜绝了模型直接吃下整份大文档所造成的长文本灾难。
- `execute_probe_query` (数据表快速批量探针):
 - **工作原理**: 基于 DuckDB 极速只读分析引擎, 对 context 下的 CSV、JSON 以及 SQLite 数据文件进行统一 of SQL 探查。CSV/JSON 文件可直接作为虚拟表 (以文件名或资产路径) 执行 SELECT 或 WITH 关联查询, SQLite 表也可直接访问。
 - **批量查询优化 (Batching Rule)**: 参数 `queries` 接收一个 `list[str]` 查询列表。系统强制要求 Agent 在进行数据确认或多表验证时, 必须将多个互不依赖 of 探查 (例如多个表的 COUNT 数量、列的 DISTINCT 值分布、样本行查询等) 打包到单次调用中一并执行。这极大地压缩了模型与系统环境的往返时延 (RTT), 提升了探测效率。
 - **返回限制**: 返回行数受内部逻辑以及 `limit` (最大 200 行) 的限制。
- `get_column_distinct_values` (单列特征值分布探查):
 - **工作原理**: 获取指定表某一数据列中按出现频率降序排列的最频繁的特征值列表 (Top N Distinct Values)。
 - **高效缓存策略**: 为了将计算开销降到最低, 系统会首先尝试从全局数据字典 `semantic_catalog` 的预计算缓存中拉取结果。当请求的特征值数量在缓存容量 (在 `configs/docker.yaml` 中由

`catalog_top_distinct_values` 配置为 `5`) 以内时, 可以直接秒回; 只有在请求超出前 5 项时, 才会下发到数据库引擎实时查表计算, 为字段基数和类别分布检查提供了快速通道。

5. 工具调用返回结果的截断处理

为了极致控制上下文窗口, 防御由于数据量巨大或代码死循环输出所导致的 Token 炸毁现象, 工具注册表 (`ToolRegistry.format_result`) 在返回结果给 Agent 前进行了统一的物理过滤和安全截断:

- **最大列表项限制:** 结合配置 `tool.max_list_items: 200`。如果工具返回的数据是列表结构 (如 SQL 探针查询返回的数据行、大纲列表等), 则强制只切片取前 200 项, 其余直接舍弃, 并递归对其进行截断处理。
 - **最大输出 Token 限制:** 结合配置 `tool.max_output_tokens: 10000`。对于列表截断后或原本就是字符串的返回内容, 如果它的 Token 长度大于 10000, 则使用底层的 Token 计数工具截取指定数量之内的 Token。
 - **友好提示后缀:** 任何被截断处理的内容末尾都会被追加一段人性化的提示后缀 (`... (内容已被截断, 返回过长。如未获取到所需信息, 请尝试其他方式而非直接读取全部文档。)`), 明确告诉 Agent 发生了截断, 需通过调整查询条件 (如添加 `LIMIT`) 来精确拉取数据。
-

6. 内置本地 Qwen3.5-35B-A3B Tokenizer 缓存计数

- **存在路径:** `src/data_agent_baseline/token_utils.py`
 - **工作机制与离线优先策略:**
 - 针对大模型的长短文本截断和输入管理, 系统放弃了字符数估算 (如 1 单词 $\approx$ 1.3 字符) 的模糊方式, 全面改用基于与生产模型一致 of Qwen3.5 Tokenizer 进行精确的 Token 级计数。
 - **离线本地缓存机制:** 程序优先寻找本地的 `assets/huggingface/Qwen3.5-35B-A3B` 分词器。如果存在, 系统会自动在引入前设定 `HF_HUB_OFFLINE=1` 和 `TRANSFORMERS_OFFLINE=1` 环境变量, 屏蔽一切可能导致网络超时报错的 Hub 在线检测; 若不存在, 再回退到线上的 Hugging Face 库获取。
 - **单例与线程安全加载:** 使用了单例模式结合线程锁 (`threading.Lock()`), 在第一次调用 `_get_tokenizer()` 时按需加锁加载 `AutoTokenizer`, 并在随后的全局 `count_tokens` 与 `truncate_by_tokens` 处理中共享同一个缓存分词器实例, 实现了极佳的计数和截断性能。
-

7. LLM 初始化请求参数调优

```
request_kwargs: dict[str, object] = {
    "model": model,
    "base_url": api_base.rstrip("/"),
    "api_key": api_key,
    "temperature": temperature,
    "top_p": 0.8,
    "extra_body": {
        "repetition_penalty": 1.05,
    },
    "max_retries": 3,
}
```

8. 模型历史思维链（Reasoning）滑动窗口管理与截断

- 相关配置: `agent.strip_reasoning_history: false` & `agent.reasoning_history_limit: 10`
- 具体实现逻辑 (`_prepare_messages_for_model`):
 - 在准备模型请求时, 系统会从最新消息往前遍历历史对话。
 - 最近 10 条带有 Reasoning 的 AI 消息会完整保留, 用来维持短期推理上下文。
 - 更早的 AI 消息会移除 Reasoning 内容, 只保留最终回复或工具调用, 减少历史上下文占用。
- 设计意义与改进点:
 - **防御 Context 爆炸**: 由于大模型的 Reasoning（深度思考内容）通常篇幅巨大（每一步往往多达数千字），如果将历史每一步的思考链无限累加，对话上下文的 Token 消耗会呈几何级数急剧飙升，最终导致耗尽 API 额度，甚至是撑爆窗口（Context Limit Exceeded）而引发运行时死机。
 - **短期工作记忆平衡**: 如果粗暴地完全抹除 (`strip_reasoning_history: true`)，模型可能由于失去刚刚计算的中间结论而导致逻辑断档。采用**滑动窗口截断机制**（保留最近 10 次的思维上下文），精妙地在“让 Agent 具备足够连贯的短期工作记忆”和“控制上下文长度以大幅节省 Token 成本并防止过载”之间找到了最佳平衡点。